WELT 0 · DOPPELSTUNDE 6 · PAPIER-COMPILER 0

W0·5 – Die Methoden-Werkstatt: Von der Aufgabe zur Methode

Heute arbeitest du fast nur mit Stift · Papier-Compiler 0 · 🎯 Im Lab: **Level 0.6** · Datei: `AQ_W0_Akademie_Geruest.txt` (Klasse Werkstattpruefung)

Die Aufnahmeprüfung der Akademie rückt näher. Dort – und im Abitur – liegt vor dir nur ein Aufgabentext: „**Implementieren Sie die Methode …**“. Kein Compiler sagt dir, dass ein `return` fehlt, kein Test meldet die vergessene leere Reihung. In der **Methoden-Werkstatt** lernst du deshalb ein festes Vorgehen, mit dem du jede Implementierungsaufgabe sicher löst – und du lernst die Fehler kennen, die in Prüfungen am häufigsten BE kosten.

Infoteil 1: Das Rezept – von der Aufgabe zur Methode

Schritt	Was tust du?	Ergebnis
1 Lesen und markieren	Was geht hinein? Was kommt heraus? Welche Bedingungen gelten (echt größer oder größer gleich)? Steht dort „neue Reihung“ oder „verändert“? Welche Sonderfälle nennt der Text?	markierter Text
2 Signatur → Kopf	Signatur in den Java-Methodenkopf übersetzen (AB 4, Infoteil 2).	Kopf
3 Randfälle und Beispiele	Eine kleine Tabelle Eingabe → erwartete Ausgabe: Normalfall, leere Reihung, ein Element, Gleichheit an der Grenze, negative Werte.	Testtabelle
4 Idee und Struktogramm	Welche Bausteine stecken drin (Zählen, Summe, Maximum, Suche, neue Reihung)? Ablauf skizzieren oder als Struktogramm zeichnen.	Plan
5 Java auf Papier	Sauber einrücken, Klammern paarweise schließen, Variablen vor Gebrauch deklarieren.	Methode
6 Abi-Check	Die Checkliste unten Punkt für Punkt durchgehen; die Testtabelle aus Schritt 3 von Hand durchspielen.	geprüfte Methode
7 Papier-Compiler	Unverändert abtippen, Tests starten, jeden Fehler ins Fehlerlogbuch.	Gewissheit

Abi-Check (Schritt 6)

- ☐ Kopf wie gefordert: Name, Parameter in Reihenfolge und Typ, Rückgabetyp („nichts“ = `void`).
- ☐ **Rückgabe in jedem Pfad** – auch wenn eine Schleife ohne Treffer endet.
- ☐ Vor jedem Zugriff auf `r[0]`, `r[r.length - 1]` oder `s.charAt(0)`: **Leer-Abfrage**.
- ☐ Schleifengrenzen: Index von **0 bis length – 1**; bei Nachbarn (`i + 1`, `i - 1`) Grenzen anpassen.
- ☐ Original **unverändert**, wenn gefordert („neue Reihung“ → `new`).
- ☐ Kommazahl, wo eine Kommazahl verlangt ist – keine ganzzahlige Division.
- ☐ Nur erlaubte Zeichenketten-Operationen; Zeichenketten mit `equals` vergleichen.
- ☐ Unsichere Stellen kurz kommentieren (`// …`) – das zeigt deine Absicht.

🎯 Im Lab üben: Level 0.6 „Die Methoden-Werkstatt: Fehlergalerie und Abi-Check“ – Punkte gibt es dort automatisch

Passend zu Rezept, Fehlergalerie F1–F10 und den Aufgaben 1–5: Fehler-Bingo, Randfälle, Abi-Check, Bug-Hunt und Code-Puzzle – jede Antwort wird sofort geprüft. Danach wartet auf der Weltkarte der Online-Boss „Aufnahmeprüfung: Die Bibliothek der Akademie“.

Das Rezept in Aktion – ein vorgemachtes Beispiel

Aufgabe (im Stil des Abiturs)

Implementieren Sie die Methode `laengsteSerie(r: Reihung von Ganzzahl, x: Ganzzahl): Ganzzahl`. Sie liefert die **Länge der längsten Folge direkt aufeinanderfolgender Elemente** von r, die alle den Wert **x** haben. Kommt x **nicht vor** oder ist r **leer**, liefert sie **0**. Die Reihung wird nicht verändert.

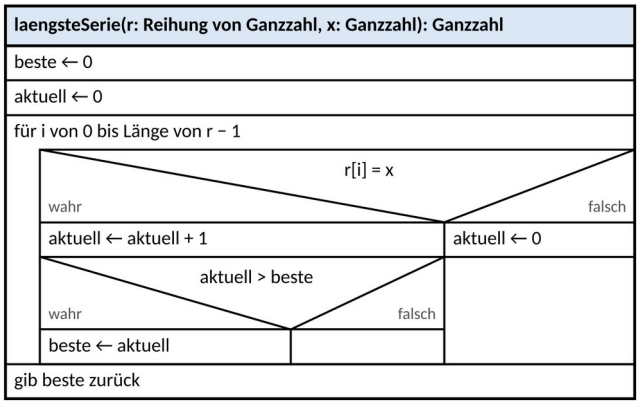
Schritt 1 – markieren: hinein: eine Reihung und eine Zahl x · heraus: eine Ganzzahl (eine Länge) · „direkt aufeinanderfolgend“: Eine Serie bricht bei jedem anderen Wert ab · Sonderfälle: x kommt nicht vor, r leer → 0 · r nur lesen.

Schritt 2 – Kopf: `public static int laengsteSerie(int[] r, int x)`

Schritt 3 – Randfälle und Beispiele (mit x = 3):

r	{3, 3, 1, 3, 3, 3, 2}	{ }	{1, 2}	{3}	{1, 3, 3}
erwartet	3	0	0	1	2 (Serie am Ende!)

Schritt 4 – Idee: zwei Zähler. `aktuell` zählt die laufende Serie und wird bei einem anderen Wert auf 0 gesetzt; `beste` merkt sich den Rekord (Baustein Maximum). Wichtig: Den Rekord **sofort** prüfen, nicht erst beim Abbruch der Serie – sonst geht eine Serie am Ende verloren.




```
public static int laengsteSerie(int[] r, int x) {
    int beste = 0;
    int aktuell = 0;
    for (int i = 0; i < r.length; i++) {
        if (r[i] == x) {
            aktuell = aktuell + 1;
            if (aktuell > beste) {
                beste = aktuell;
            }
        } else {
            aktuell = 0;
        }
    }
    return beste;
}
```

Schritt 5 – Java: siehe oben rechts. **Schritt 6 – Abi-Check:** Kopf ✓ · `return beste` nach der Schleife, also in jedem Pfad ✓ · kein Zugriff auf `r[0]`, darum keine Leer-Abfrage nötig – die leere Reihung liefert 0 ✓ · Index 0 bis length − 1 ✓ · r wird nur gelesen ✓. Testtabelle von Hand für {3, 3, 1, 3, 3, 3, 2}:

i	0	1	2	3	4	5	6
r[i]	3	3	1	3	3	3	2
aktuell	1	2	0	1	2	3	0
beste	1	2	2	2	2	3	3

Merke: Die Testtabelle aus Schritt 3 ist kein Schmuck. Wer `beste` nur im else-Zweig aktualisiert, liefert für {1, 3, 3} den Wert 0 – das fällt nur auf, wenn man den Fall „Serie am Ende“ wirklich durchspielt.

Infoteil 2: Die Fehlergalerie

Diese zehn Fehler kosten in Prüfungen die meisten BE. Präge dir ihre **Merkmale** ein – in Aufgabe 1 musst du sie wiedererkennen.

F1 Fehlender Rückgabepfad

FALSCH

```
public static int indexVonNull(int[] r) {
    for (int i = 0; i < r.length; i++) {
        if (r[i] == 0) {
            return i;
        }
    }
}   // Compiler: missing return statement
```

RICHTIG

```
public static int indexVonNull(int[] r) {
    for (int i = 0; i < r.length; i++) {
        if (r[i] == 0) {
            return i;
        }
    }
    return -1;   // kein Treffer
}
```

Woran erkennst du ihn? Frage dich: Was liefert die Methode, wenn die Schleife ohne Treffer endet oder kein if zutrifft?

F2 Off-by-one: eine Runde zu viel oder zu wenig

FALSCH

```
int summe = 0;
for (int i = 0; i <= r.length; i++) {
    summe = summe + r[i];   // Absturz!
}
```

RICHTIG

```
int summe = 0;
for (int i = 0; i < r.length; i++) {
    summe = summe + r[i];
}
```

Woran erkennst du ihn? `<=` mit `length`, Start bei 1 statt 0, `r[i + 1]` ohne angepasste Grenze. Spiele den ersten und den letzten Durchlauf durch.

F3 = statt == (Zuweisung statt Vergleich)

FALSCH

```
if (gefunden = true) {   // immer wahr!
    return "ja";
}
```

RICHTIG

```
if (gefunden) {          // == true geht auch
    return "ja";
}
```

Woran erkennst du ihn? Bei Wahrheitswerten kompiliert `=` in der Bedingung und ist immer erfüllt; bei Zahlen meldet der Compiler einen Fehler.

F4 Ganzzahlige Division

FALSCH

```
int treffer = 3;
int versuche = 4;
double anteil = treffer / versuche * 100;
// anteil ist 0.0
```

RICHTIG

```
int treffer = 3;
int versuche = 4;
double anteil = 100.0 * treffer / versuche;
// anteil ist 75.0
```

Woran erkennst du ihn? Zwei Ganzzahlen werden mit `/` geteilt, das Ergebnis soll eine Kommazahl sein. Lösung: Cast **vor** der Division oder eine Kommazahl beteiligen.

F5 Parameter überschrieben

FALSCH

```
public static int summeAb(int[] r, int start) {
    int summe = 0;
    for (start = 0; start < r.length; start++) {
        summe = summe + r[start];
    }
    return summe;   // ignoriert start
}
```

RICHTIG

```
public static int summeAb(int[] r, int start) {
    int summe = 0;
    for (int i = start; i < r.length; i++) {
        summe = summe + r[i];
    }
    return summe;
}
```

Woran erkennst du ihn? Einem Parameter wird ein fester Wert zugewiesen, bevor er benutzt wurde. Parameter sind Eingaben – lies sie, statt sie zu überschreiben.

F6 Rückgabe in der Schleife zu früh

FALSCH

```
for (int i = 0; i < r.length; i++) {
    if (r[i] > 0) {
        return true;
    } else {
        return false;   // prüft nur r[0]
    }
}
return true;
```

RICHTIG

```
for (int i = 0; i < r.length; i++) {
    if (r[i] <= 0) {
        return false;   // Gegenbeispiel gefunden
    }
}
return true;            // erst nach der Schleife
```

Woran erkennst du ihn? Ein `return` in **beiden** Zweigen innerhalb der Schleife: Die Schleife läuft nur einmal. „Alle …“ ist erst nach der Schleife sicher.

F7 Reihung verändert statt Kopie

FALSCH

```
public static int[] negiert(int[] r) {
    int[] erg = r;   // nur die Referenz!
    for (int i = 0; i < r.length; i++) {
        erg[i] = -r[i];
    }
    return erg;   // r ist jetzt auch negiert
}
```

RICHTIG

```
public static int[] negiert(int[] r) {
    int[] erg = new int[r.length];
    for (int i = 0; i < r.length; i++) {
        erg[i] = -r[i];
    }
    return erg;
}
```

Woran erkennst du ihn? „liefert eine neue Reihung“ ohne `new` in der Methode.

F8 == statt equals bei Zeichenketten

FALSCH

```
if (woerter[i] == wort) {
    anzahl++;
}
```

RICHTIG

```
if (woerter[i].equals(wort)) {
    anzahl++;
}
```

Woran erkennst du ihn? `==` zwischen zwei Zeichenketten vergleicht Referenzen. Die Online-IDE verzeiht das, echtes Java und das Abitur nicht.

F9 Leer-Abfrage vergessen

FALSCH

```
public static char erstesZeichen(String s) {
    return s.charAt(0);   // Absturz bei ""
}
```

RICHTIG

```
public static char erstesZeichen(String s) {
    if (s.length() == 0) {
        return ' ';
    }
    return s.charAt(0);
}
```

Woran erkennst du ihn? Zugriff auf Index 0 oder auf das letzte Element ohne vorherige Prüfung der Länge.

F10 Falscher Startwert bei Maximum/Minimum

FALSCH

```
int max = 0;
for (int x : r) {
    if (x > max) {
        max = x;      // {-4, -2, -9} liefert 0
    }
}
```

RICHTIG

```
int max = r[0];     // vorher: Leer-Abfrage!
for (int x : r) {
    if (x > max) {
        max = x;
    }
}
```

Woran erkennst du ihn? Der Startwert ist eine feste Zahl statt eines Elements der Reihung. 0 ist falsch, sobald alle Werte negativ (bzw. beim Minimum alle positiv) sind.

Aufgabe 1: Fehler-Bingo

PAPIER ★

Jeder Schnipsel enthält **einen** Fehler aus der Galerie. Trage die Fehlernummer (F1 bis F10) ein und korrigiere die Stelle. Wer zuerst eine Zeile, Spalte oder Diagonale im Bingo-Feld unten voll hat, ruft „Bingo!“ – gewertet wird aber nur mit richtiger Korrektur.

Nr.	Schnipsel (Absicht in Klammern)	F-Nr.	Korrektur

S1	`int summe = 0;  // (Summe aller Werte)`
	`for (int i = 1; i < r.length; i++) {`
	`  summe = summe + r[i];`
	`}`		

S2	`if (name == "Aria") {  // (Name prüfen)`
	`  bonus = 10;`
	`}`		

S3	`int richtige = 7;`
	`int gesamt = 8;`
	`double prozent = richtige / gesamt * 100;`		

S4	`for (int wert : r) {  // (kommt x vor?)`
	`  if (wert == x) {`
	`  return true;`
	`  } else {`
	`  return false;`
	`  }`
	`}`
	`return false;`		

S5	`public static int max(int a, int b) {`
	`  if (a > b) { return a; }`
	`  if (b > a) { return b; }`
	`}`		

S6	`int kleinstes = 0;  // (Minimum von r)`
	`for (int x : r) {`
	`  if (x < kleinstes) { kleinstes = x; }`
	`}`		

S7	`int[] kopie = original; // (original soll bleiben)` `kopie[0] = 99;`		

S8	`if (fertig = false) {  // (fertig ist boolean)`
	`  weiter();`
	`}`		

S9	`public static int erstes(int[] r) {`
	`  return r[0];  // (leere Reihung: -1)`
	`}`		

S10	`public static String wiederhole(String text, int anzahl) {`
	`  anzahl = 3;  // (anzahl-mal wiederholen)`
	`  String e = "";`
	`  for (int i = 0; i < anzahl; i++) { e = e + text; }`
	`  return e;`
	`}`		

Bingo-Feld: Trage die Schnipsel-Nummern ein, deren Fehler du gefunden und korrigiert hast.

F1	F2	F3	F4	F5

F6	F7	F8	F9	F10

Aufgabe 2: Papier-Compiler 0: Die Werkstattprüfung

PAPIER → IDE ★★★

Regeln des Papier-Compilers

1. Schreibe die Methode **vollständig von Hand** – ohne Gerät, ohne Spickzettel außer den Werkzeugkarten.
2. Gehe den Abi-Check durch (Infoteil 1).
3. Tippe deine Lösung **unverändert** ab – Zeichen für Zeichen, auch mit Fehlern!
4. Starte `PapierCompiler0Test`. **Alle** Tests grün beim ersten Lauf: Deine Handschrift „compiliert" – genau das zählt im Abitur. Sonst: erst ins Fehlerlogbuch, dann korrigieren.
5. Jeder Fehler kommt ins Fehlerlogbuch – erst dann korrigierst du in der IDE.

Aufgabe (wie im Abitur)

Implementieren Sie die Methode `anzahlUeberDurchschnitt(r: Reihung von Ganzzahl): Ganzzahl`. Sie liefert die Anzahl der Werte in r, die **echt größer** als der Durchschnitt aller Werte von r sind. Der Durchschnitt ist als **Kommazahl** zu berechnen. Für eine **leere Reihung** liefert die Methode 0. Die Reihung wird nicht verändert.

Bewertung: [5 BE]

a) **Schritt 1 und 2:** Markiere im Aufgabentext die Bedingungen und Sonderfälle. Schreibe den Java-Methodenkopf.

b) **Schritt 3:** Ergänze die erwarteten Ergebnisse.

r	{12, 7, 15, 9, 3}	{5, 5, 5}	{ }	{-1, -2}	{7}
Durchschnitt			–		
erwartet					

c) **Schritt 4 und 5:** Implementiere die Methode in Java (bei Bedarf zuerst eine Skizze oder ein Struktogramm auf einem Extrablatt).



d) **Schritt 6 – Abi-Check:** Spiele deine Methode mit {−1, −2} von Hand durch. Welchen Wert hat der Durchschnitt in deiner Lösung?

---

e) **Schritt 7 – Papier-Compiler:** Tippe die Methode **unverändert** in die Klasse `Werkstattpruefung` ab (mit Kopf). Schalte dann die beiden Blöcke frei – in der Testklasse **und** im Hauptprogramm – und starte `PapierCompiler0Test`:

```
/* PAPIER-COMPILER-0-START        <- diese ganze Zeile löschen
class PapierCompiler0Test {
    ...
}
PAPIER-COMPILER-0-ENDE */         <- diese ganze Zeile löschen
```

Ergebnis: ☐ alle 6 Tests grün beim ersten Lauf ☐ grün nach Korrektur – Anzahl der Korrekturen: ____

Fehlerlogbuch

Nr.	Was war falsch?	Art: Syntax / Logik / Abi-Check	So vermeide ich es
1			
2			
3			

Aufgabe 3–5: Drei Abi-Aufgaben zur Wahl

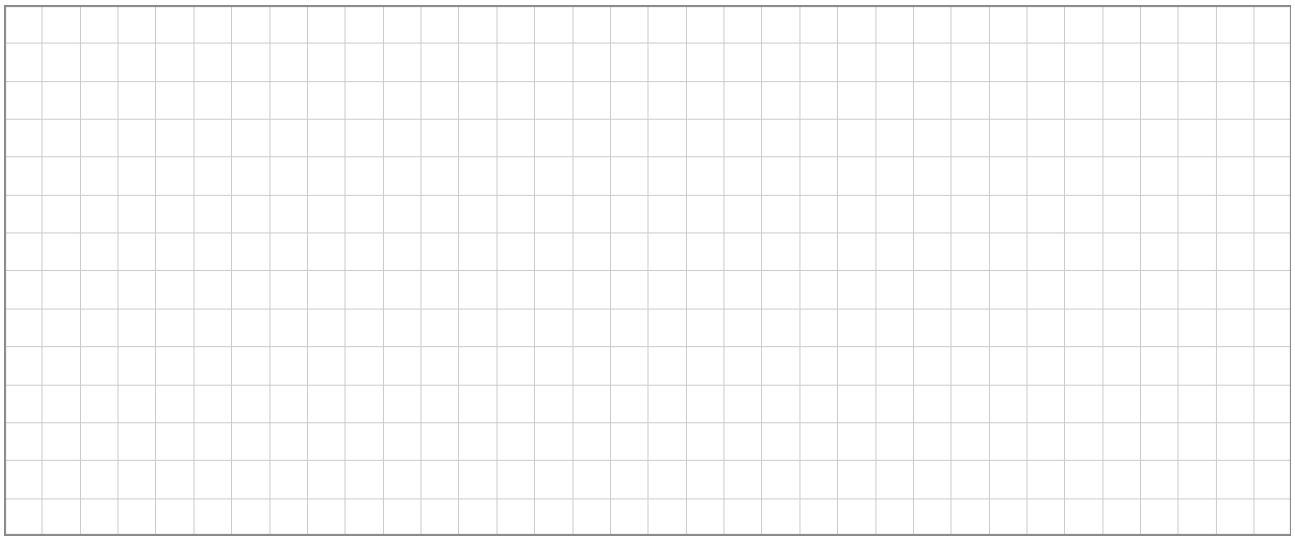
PAPIER ★★ – ★★★

Wähle **mindestens eine** Aufgabe und bearbeite sie nach dem Rezept – ohne Gerät, mit Zeitlimit (≈ 2,5 min pro BE). Die übrigen sind dein Boss-Training. Notiere oben jeweils Kopf und Randfälle (Schritte 2 und 3).

Aufgabe 3: Die Treppe der Akademie ★★ [4 BE]

Implementieren Sie die Methode `istTreppe(r: Reihung von Ganzzahl): Wahrheitswert`. Sie liefert wahr, wenn jedes Element von r (ab dem zweiten) **genau um 1 größer** ist als sein Vorgänger, z. B. für {4, 5, 6, 7}. Für Reihungen mit weniger als zwei Elementen liefert sie wahr.

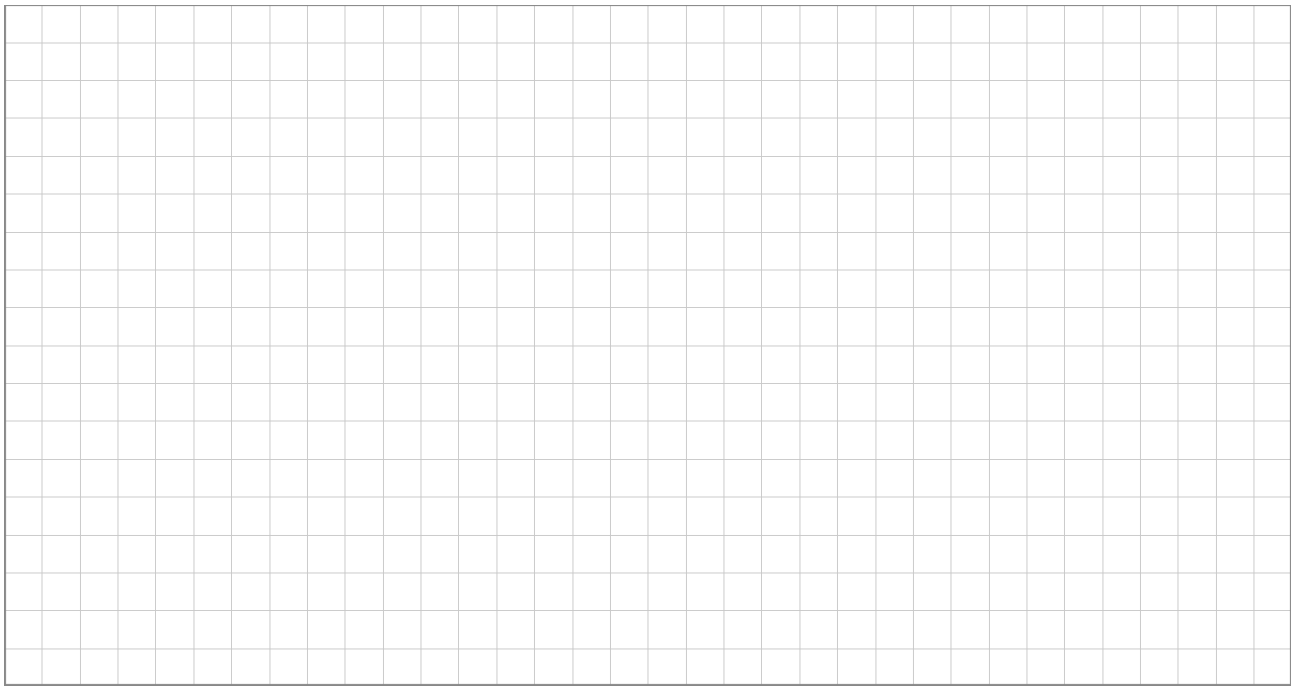
Kopf und Randfälle:



Aufgabe 4: Das Zeichen im Zauberbuch ★★★ [5 BE]

Die Methode `anzahlMit(woerter: Reihung von Zeichenkette, c: Zeichen): Ganzzahl` liefert die Anzahl der Wörter in woerter, in denen das Zeichen c **mindestens einmal** vorkommt (Groß- und Kleinschreibung werden unterschieden). Implementieren Sie zunächst eine geeignete Hilfsmethode `enthaeltZeichen(s: Zeichenkette, c: Zeichen): Wahrheitswert` und anschließend die Methode anzahlMit unter Verwendung dieser Hilfsmethode.

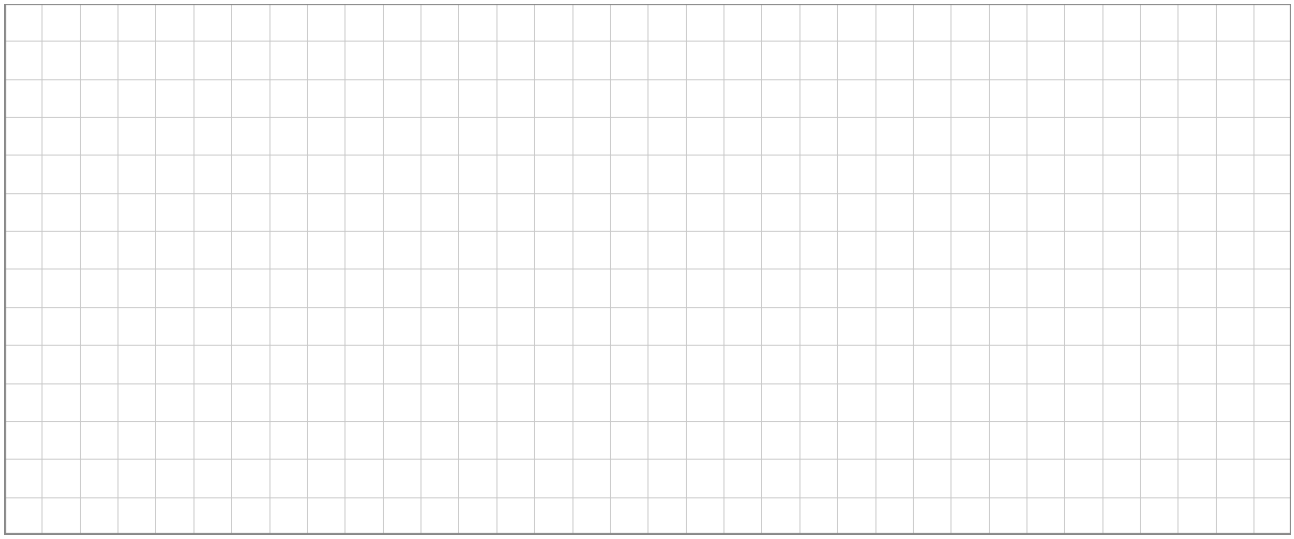
Kopf und Randfälle:



Aufgabe 5: Ohne Rand ★★ [4 BE]

Implementieren Sie die Methode `mitte(r: Reihung von Ganzzahl): Reihung von Ganzzahl`. Sie liefert eine **neue** Reihung, die alle Elemente von r außer dem ersten und dem letzten in unveränderter Reihenfolge enthält, z. B. {8, 1} für {3, 8, 1, 9}. Hat r weniger als drei Elemente, wird eine leere Reihung (Länge 0) geliefert. r bleibt unverändert.

Kopf und Randfälle:



Aufgabe 6: Peer-Korrektur mit dem Kernpunkt-Bogen

PAPIER ★★ Rolle: Helfer:in

Tauscht eure Lösungen zu Aufgabe 3, 4 oder 5 **teamübergreifend**. Die Lehrkraft zeigt nach der Schreibphase die **Kernpunkte** der Aufgabe – erst dann wird korrigiert. Bewertet wird nur, was auf dem Papier steht, nicht, was gemeint war.

Kernpunkt-Bogen zu Aufgabe ____	erfüllt? (✓ / × / teilweise)	Hinweis an die Autorin / den Autor
Kopf exakt wie in der Signatur (Name, Parameter, Rückgabetyp)		
Randfälle laut Aufgabentext korrekt und ohne Laufzeitfehler		
Schleifengrenzen korrekt (kein Off-by-one)		
Rückgabe in jedem Pfad, richtiger Typ		
Original unverändert bzw. neue Reihung korrekt (falls gefordert)		
Hilfsmethode verwendet bzw. erlaubte Operationen und equals		
Kernpunkt 1 (Lehrkraft):		
Kernpunkt 2 (Lehrkraft):		
Kernpunkt 3 (Lehrkraft):		

Vorschlag: ____ von ____ BE Korrigiert von (Name): _____

Tipp: Eine Rückmeldung, die **erklärt** statt vorsagt („Spiel mal {1, 3, 3} durch – was liefert deine Methode?“), ist ein echter Helfer:in-Einsatz – ohne Punkte, aber genau das schärft den Blick für Abi-Fehler. Fehler, die du bei dir selbst entdeckst, kommen ins Fehlerlogbuch.

Mein Fehlerlogbuch – Methoden-Werkstatt

Trage jeden Fehler aus Papier-Compiler, Peer-Korrektur und Bingo ein, den du **selbst** gemacht hast. Ordne ihn einer Nummer der Fehlergalerie zu (oder „neu“). Vor dem Boss-Kampf liest du diese Seite noch einmal.

Nr.	Aufgabe	Fehler (F-Nr. oder neu)	Was war falsch?	So vermeide ich es
1				
2				
3				
4				
5				
6				